



Academic Year **2025/2026**
Semester **Spring**
Academic Level: **Level 2**
Revision Coding Questions

Subject: (ECT 372) Computer Design & Architecture

Assembly Revision Questions

1. Variable Definition & Initialization

Question: Write the .data segment to define the following variables:

- A 32-bit signed integer named val1 with value -50.
- A 16-bit unsigned integer named val2 with value 2500.
- An uninitialized array named buffer of 100 bytes.
- A string constant named msg containing "Welcome" followed by null terminator.

Answer:

```
.data  
val1  SDWORD -50  
val2  WORD 2500  
buffer BYTE 100 DUP(?)  
msg   BYTE "Welcome", 0
```

2. Array Size Calculation

Question: Define an array of 5 WORDs containing values 10, 20, 30, 40, 50. Then, write a constant declaration (using =) named ArrayCount that automatically calculates the number of elements in this array using the current location counter \$.

Answer:

```
.data  
myArray  WORD 10, 20, 30, 40, 50  
ArrayCount = ($ - myArray) / 2
```

3. Register Swapping

Question: Write a to swap the values of two 32-bit variables, varA and varB, using the EAX and EBX registers.

Answer:

```
mov eax, varA
mov ebx, varB
mov varA, ebx
mov varB, eax
```

5. Simple Arithmetic Expression

Question: Translate the following mathematical expression into Assembly code using EAX as the accumulator: $R = (A + B) - (C + 5)$. Assume A, B, C, and R are 32-bit variables.

Answer:

```
mov eax, A
add eax, B    ; EAX = A + B
sub eax, C    ; EAX = (A + B) - C
sub eax, 5    ; EAX = (A + B) - C - 5
mov R, eax    ; Store result
```

6. Increment and Decrement

Question: Write a program that asks the user to enter an integer. Print the integer immediately preceding it and the integer immediately following it.

Answer:

```
call ReadInt    ; Read value into EAX
mov ebx, eax    ; Save a copy
dec eax        ; Calculate Predecessor
call WriteInt   ; Print Predecessor
call Crlf
mov eax, ebx    ; Restore original
inc eax        ; Calculate Successor
call WriteInt   ; Print Successor
```

7. Negative and Two's Complement

Question: Write a snippet that takes a positive integer stored in EAX, converts it to its negative equivalent using 2's complement logic (without using SUB), and then prints it.

Answer:

```
neg eax        ; Converts value in EAX to its 2's complement (Negative)
call WriteInt
```

8. 16-bit Addition with Carry

Question: You have two 16-bit variables, num1 and num2. Move num1 into AX, add num2 to it, and print the resulting value as a hexadecimal number.

Answer:

```
mov ax, num1
add ax, num2
call WriteHex ; Displays AX in Hex
```

9. Uppercase Conversion

Question: Write a loop that iterates through the string str1 BYTE "hello",0 and converts it to uppercase "HELLO" using the AND instruction.

Answer:

```
mov esi, OFFSET str1
mov ecx, LENGTHOF str1 - 1 ; Exclude null terminator
L1:
    and BYTE PTR [esi], 11011111b ; Clear Bit 5 to convert to Upper
    inc esi
    loop L1
mov edx, OFFSET str1
call WriteString
```

11. Setting and Clearing Bits

Question: Given a variable flags BYTE 00000000b:

1. Set Bit 4 and Bit 7 using OR.
2. Clear Bit 4 using AND.
3. Toggle Bit 0 using XOR.

Answer:

```
or flags, 10010000b ; Set bits 7 and 4
and flags, 11101111b ; Clear bit 4 (Mask 0 at bit 4)
xor flags, 00000001b ; Toggle bit 0
```

12. Simple Encryption

Question: Encrypt a character in AL by XORing it with the key 55h. Then, decrypt it back to the original value. Show the code.

Answer:

```
xor al, 55h    ; Encrypt
xor al, 55h    ; Decrypt (XORing twice returns original value)
```

13. Grade Pass/Fail

Question: Ask the user for a grade (0-100). If the grade is greater than or equal to 60, print "Pass"; otherwise, print "Fail".

Answer:

```
call ReadInt
cmp  eax, 60
jae  Passed    ; Jump if Above or Equal
mov  edx, OFFSET msgFail
jmp  Print
Passed:
mov  edx, OFFSET msgPass
Print:
call WriteString
```

14. Comparing Signed Integers

Question: Compare two signed integers in EAX and EBX. Jump to the label Greater if EAX > EBX. (Hint: Use the correct jump instruction for signed data).

Answer:

```
cmp  eax, ebx
jg   Greater    ; Jump if Greater (Signed comparison)
```

15. Range Checking (0-9)

Question: Read a character into AL. Check if it is a digit (ASCII '0' to '9'). If it is a digit, print "Yes"; otherwise, print "No".

Answer:

```
call ReadChar
cmp al, '0'
jb  NotDigit    ; Below '0' -> No
cmp al, '9'
ja  NotDigit    ; Above '9' -> No
mov edx, OFFSET msgYes
jmp Done
NotDigit:
mov edx, OFFSET msgNo
Done:
call WriteString
```